Proyecto Mini Trello.

PROYECTO MARGARET

Programación de Sistemas Informáticos.

[Nombre del autor]
01-03-2026

ÍNDICE

1. – Descripción 2
2. – Análisis del Mercado Actual 4
3. -Propuesta innovadora 6
4. -Estudio de Viabilidad 8
5. -Arquitectura de la Aplicación 10
6. Futuras Mejoras y conclusiones 14
7. -Manual usuario 16
8. -Bibliografía 18

1.-Descripción

Margaret está inspirado en un gestor de eventos, el cual vinculado a una base de datos, registra y guarda el usuario, email y la contraseña, y da la opción de crear tareas y programarlas como pendientes o completadas.

Es una página web, la cual está vinculada a un código de programación con Python en Visual Studio Code en el cual se utiliza un framework ligado a Flask, dónde se ejecuta. Vincula además a la base de datos donde tenemos la tabla usuario y tarea para guardar los respectivos registros.

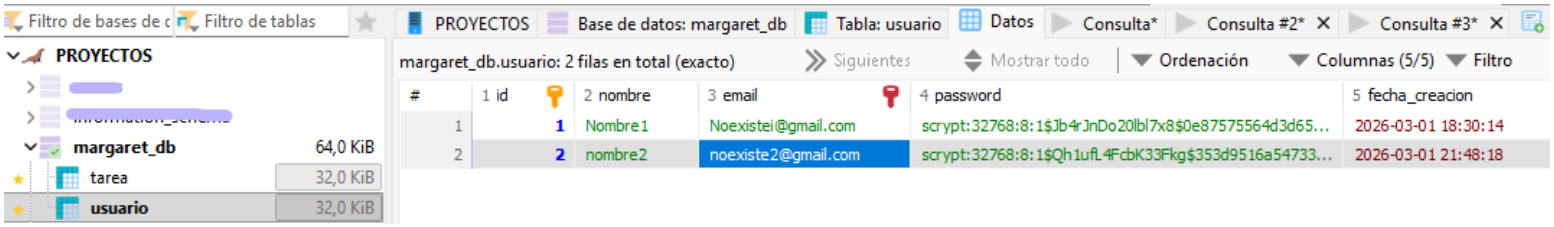
- Motor (Backend): Python procesa las solicitudes, valida las credenciales y gestiona la lógica de las tareas.
- Interfaz (Frontend): HTML, "bonita y agradable", permitiendo cambios de estado (pendiente/completado) sin recargar toda la página.
- Almacenamiento (Base de Datos): SQL es perfecto aquí por su naturaleza de archivo único, lo cual facilita enormemente las copias de seguridad de la oficina.

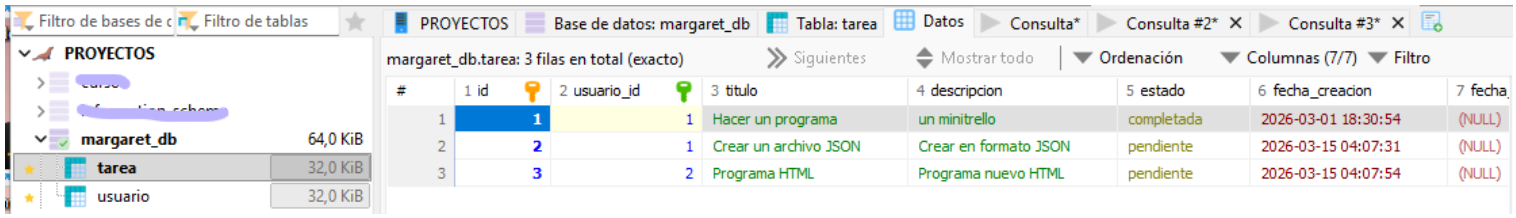
- Gestión de Roles (Admin vs. Usuario)

La distinción de permisos es vital para que Margaret sea un gestor serio:

- Administrador: Tiene el control total. Puede gestionar la creación de cuentas de usuario, ver estadísticas globales de la oficina y realizar mantenimientos en la base de datos.

- Usuario: Tiene un espacio de trabajo limpio donde solo ve sus tareas asignadas. Su enfoque es la productividad: crear, marcar como completado y organizar su día.





2.-Análisis del Mercado Actual

NoemiTelecom.com

Empresa NoemiTelecom.com tiene la interfaz de Margaret para sus datos personales.

Margaret está dirigida a empresas que deseen utilizar un gestor de tareas con su propia base de datos, de forma local y con el puerto cerrado.
Básicamente, se trata de un mini Trello personal para una oficina que desee guardar sus datos de forma que no sea posible acceder a ellos gracias a su privacidad total. Garantiza un aislamiento total.

Soberanía de datos: Los datos residen exclusivamente en hardware propiedad de la empresa, dentro de su perímetro físico.

Seguridad mediante aislamiento: Al operar con el **puerto cerrado**, se elimina la superficie de ataque externa. La aplicación es inaccesible desde Internet, haciendo inútil cualquier intento de intrusión remota convencional.

Independencia operativa: Garantiza que la operativa interna de la oficina no se vea afectada por caídas del servicio de Internet o cambios en las políticas de suscripción de terceros.

¿A qué empresas va dirigido?

Esta solución es altamente atractiva para:

Despachos legales y consultoras: Donde la confidencialidad de la información de los clientes es una obligación legal.

Departamentos de I+D y Defensa: Donde la propiedad intelectual es el activo más valioso y no puede ser expuesta en servidores compartidos.

Organizaciones con entornos regulados: Empresas que necesitan auditorías estrictas sobre dónde y cómo se almacena cada byte de información (GDPR, ISO 27001).

3.- Propuesta Innovadora

Margaret no es solo un gestor de tareas; es un puente estratégico diseñado para transformar la relación entre las startups y sus clientes. Mientras que otras herramientas del mercado se centran únicamente en la gestión interna, **Margaret** prioriza la **transparencia operativa** como su pilar fundamental.

El reto del mercado actual

En la gestión de proyectos digitales, la brecha entre desarrolladores y clientes suele generar una "zona de incertidumbre". La falta de visibilidad sobre el estado real de los avances a menudo deriva en ansiedad, desconfianza y, finalmente, conflictos innecesarios.

La solución: Transparencia total como ventaja competitiva

El diferencial de **Margaret** es su **módulo de seguimiento proactivo para el cliente**. Entendemos que el cliente no solo quiere un producto final, quiere ser parte del camino. Por ello, nuestra plataforma permite que el cliente visualice el

progreso del proyecto en tiempo real, eliminando las dudas sobre "en qué punto estamos".

Filosofía de diseño: Accesibilidad universal

Aunque Margaret es una herramienta potente, su diseño se aleja de la complejidad innecesaria. Hemos creado una **interfaz moderna, intuitiva y visualmente atractiva**, pensada para que cualquier persona —independientemente de su competencia técnica— pueda navegar por ella cómodamente.

- **Para la empresa:** Centraliza la comunicación y organiza los objetivos con precisión.
- **Para el cliente:** Elimina la incertidumbre, fomenta la confianza y reduce los roces comunicativos.

En un mercado saturado, el éxito no solo depende de la funcionalidad técnica, sino de la capacidad de mantener una relación clara y fluida con quienes depositan su confianza en nosotros. **Con Margaret, la comunicación no es un proceso secundario; es la base del proyecto.**

4.-Estudio de Viabilidad

Análisis de Viabilidad de Margaret

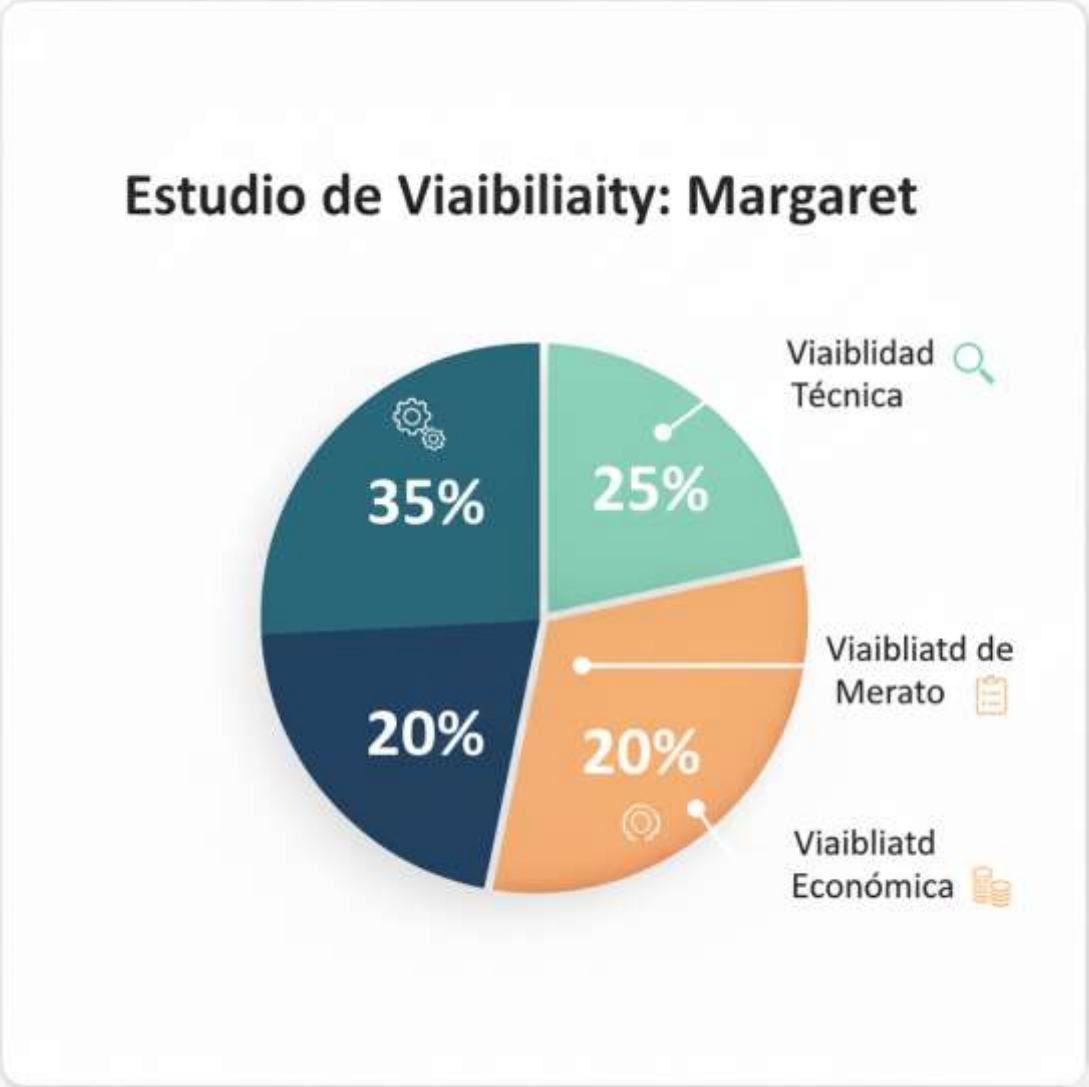
Viabilidad Técnica (35%): Margaret se basa en Python, un lenguaje robusto, y arquitectura local. Al no requerir servicios en la nube complejos (AWS/Azure), la infraestructura es simple, pero debe ser impecable en términos de cifrado local (bcrypt para contraseñas) y copias de seguridad automatizadas.

Viabilidad de Mercado (25%): Existe una demanda creciente por la "Soberanía Digital". Las empresas están cansadas de la inseguridad de la nube pública y la dependencia de suscripciones. Tu propuesta de "transparencia para el cliente" es el gancho perfecto para diferenciarte de herramientas genéricas.

Viabilidad Operativa (20%): La clave aquí es la usabilidad. Como tu objetivo es que personas no técnicas la usen, la curva de aprendizaje debe ser casi inexistente. Si la interfaz falla en ser intuitiva, el proyecto pierde valor.

Viabilidad Económica (20%): Al ser una solución on-premise, el modelo de negocio se basa en licencias perpetuas o mantenimiento/soporte anual. Esto genera un flujo de caja

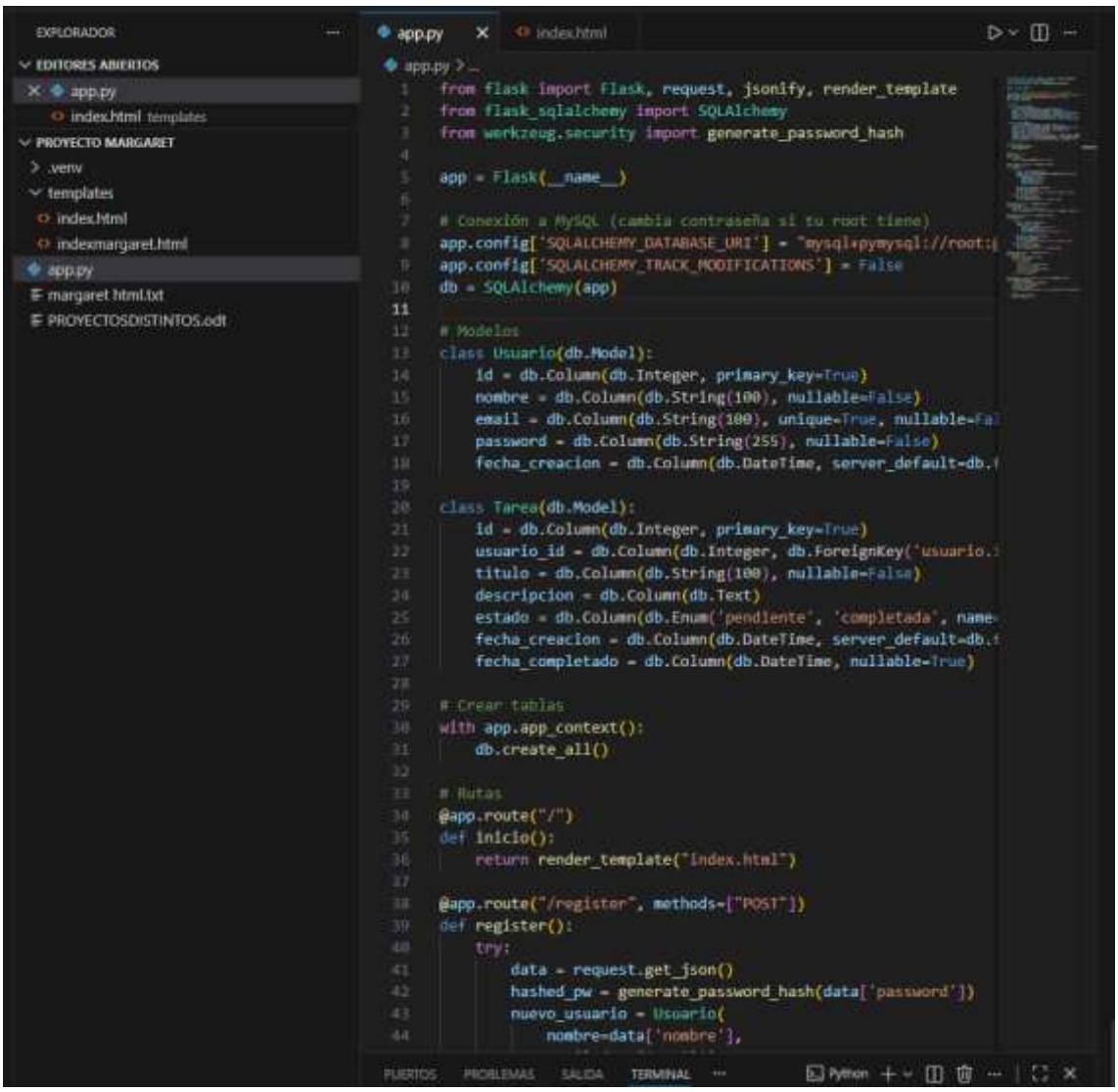
predecible y reduce los costes operativos de mantenimiento de servidores.



5.-Arquitectura de la Aplicación

Está creada con Visual Studio Code en lenguaje de programación Python, con framework Flask, el archivo para abrir con página web está creado con html en formato json, vincula la base de datos de heidisql, que se ejecuta con xampp, donde tiene activado apache2 y mysql, su base de datos es mariadb.

Visual Studio Code: código Python con flask



HTML:

app.py | index.html | indexmargaret.html

templates > index.html

```
<style>
  /* Reset básico */
  * {
    box-sizing: border-box;
  }

  body {
    font-family: 'Quicksand', sans-serif;
    background: linear-gradient(135deg, #a2d5c6 0%, #70c1
    color: #014d4d;
    margin: 40px;
    min-height: 100vh;
  }

  h1 {
    font-family: 'Parisienne', cursive;
    font-size: 4rem;
    text-align: center;
    color: #1a535c;
    margin-bottom: 15px;
    text-shadow: 1px 1px 3px rgba(255 255 255 / 0.6);
  }

  h2 {
    font-family: 'Quicksand', sans-serif;
    font-weight: 700;
    font-size: 1.8rem;
    text-align: center;
    margin-top: 40px;
    margin-bottom: 20px;
    color: #05668d;
  }

  form {
    background: rgba(255 255 255 / 0.8);
    border-radius: 18px;
    box-shadow: 0 4px 15px rgba(0 0 0 / 0.15);
    padding: 25px 30px;
    max-width: 420px;
    margin: 0 auto 35px auto;
  }

  input, textarea, select {
    font-family: 'Quicksand', sans-serif;
```

Página Margaret:

Base de datos:

```
USE margaret_db;
SHOW TABLES;
SELECT * FROM usuario;
SELECT * FROM tarea;
```

Filter ...
Columnas
Funciones SQL
Palabras clave SQL

TABLE_NAMES (2r × 1c) | usuario (2r × 5c) | tarea (3r × 7c)

#	1 id	2 usuario_id	3 titulo	4 descripcion	5 estado	6 fecha_creacion
1	1	1	Hacer un programa	un minitrello	completada	2026-03-01 18:30:54
2	2	1	Crear un archivo JSON	Crear en formato JSON	pendiente	2026-03-15 04:07:31
3	3	2	Programa HTML	Programa nuevo HTML	pendiente	2026-03-15 04:07:54

6.-Futuras mejoras y conclusiones

Para que Margaret pase de ser un gestor de tareas a un ecosistema de trabajo colaborativo, propongo las siguientes fases de escalabilidad:

- **Integración de Archivos Local:** Implementar un sistema de gestión documental cifrado dentro de la propia base de datos, permitiendo subir documentos adjuntos (PDFs, imágenes, facturas) que residan exclusivamente en el servidor local.
- **Sistema de Notificaciones Push Internas:** Añadir un servicio de avisos para cuando un usuario realice un cambio de estado en una tarea, asegurando que el equipo esté sincronizado sin necesidad de salir de la aplicación.
- **Módulo de Analítica y Reportes:** Generar gráficos automáticos (como el de viabilidad que acabamos de hacer) para que los administradores vean la carga de trabajo y el tiempo de respuesta promedio de los equipos.
- **Modo "Offline-First":** Optimizar la aplicación para que, incluso en caso de una caída total de la red interna, el usuario pueda seguir trabajando y la base de datos se sincronice automáticamente en cuanto se restablezca la conexión local.

Condiciones de Operación y Seguridad

- Al tratarse de una herramienta diseñada para funcionar con el **puerto cerrado**, es vital establecer reglas claras de uso para mantener la integridad del sistema:
- **Acceso Restringido:** El uso de Margaret está limitado exclusivamente a la red local (LAN) de la empresa. Queda terminantemente prohibido habilitar el reenvío de puertos (*port forwarding*) en el *router* para acceso remoto sin una red VPN configurada previamente por el equipo de TI.
- **Responsabilidad de Backup:** Es responsabilidad del Administrador realizar copias de seguridad de la base de datos SQLite en un medio físico externo (o almacenamiento "frío") al menos una vez por semana.
- **Actualizaciones de Seguridad:** Aunque el sistema sea local, se recomienda que el Administrador mantenga actualizadas las dependencias del entorno de Python (librerías, framework) para prevenir vulnerabilidades conocidas.
- **Política de Credenciales:** Todos los usuarios deben utilizar contraseñas con una complejidad mínima (8 caracteres, incluyendo números y caracteres especiales), gestionadas bajo la política de seguridad interna de la organización.

8.-Manual usuario

Acceso al Sistema

1. Asegúrate de estar conectado a la red local de la oficina (Wi-Fi o cable).
2. Abre tu navegador web e introduce la dirección IP interna de tu servidor Margaret (ejemplo: http://192.168.1.XX).
3. Ingresa con tu **usuario** y **contraseña** proporcionados por tu administrador.

Interfaz Principal (Dashboard)

La pantalla principal está dividida secciones clave:

- **Registrar nuevo usuario:** aquí podrás registrarte en la base de datos.
- **Crear nueva tarea:** podrás crear tantas tareas necesites.
- **Desplegable con tres opciones:** todos, pendientes y completadas. En este apartado podrás ver tus tareas.

Guía de Funciones

Cómo gestionar tus tareas

- **Regístrate si eres nuevo usuario rellenando los campos.**
- **Crear una nueva tarea, añadiendo los campos que se requieren.**
- **Ver todos:** verás todas las tareas tanto completadas como pendientes.
- **Marcar como completada:** Simplemente haz clic en la barra debajo de la descripción de la tarea. La interfaz cambiará visualmente para mostrar el progreso.
- **Visualizar completadas:** verás en el desplegable las siguientes opciones: todos, completadas o pendientes

---

Roles y Permisos

- **Usuario (Staff):** Puede crear, editar y completar sus propias tareas. Su enfoque es la productividad diaria.
- **Administrador:** Tiene acceso a la gestión de usuarios, creación de cuentas, visualización de métricas globales y mantenimiento de la base de datos.

8.- Bibliografía

1. Entorno de Desarrollo y Lenguaje:

Microsoft. (2026). Visual Studio Code User Guide. Recuperado de https://code.visualstudio.com/docs

Python Software Foundation. (2026). Python 3.12.x Documentation. Recuperado de https://docs.python.org/3/

2. Desarrollo Web (Backend y Frontend)

Pallets Projects. (2026). Flask Documentation (3.0.x). Recuperado de https://flask.palletsprojects.com/

Mozilla Contributors. (2026). HTML/CSS: Guía para desarrolladores web. MDN Web Docs. Recuperado de https://developer.mozilla.org/es/docs/Web

3. Servidor, Base de Datos y Gestión

Apache Software Foundation. (2026). Apache HTTP Server Documentation. Recuperado de https://httpd.apache.org/docs/

MariaDB Corporation. (2026). MariaDB Knowledge Base. Recuperado de https://mariadb.com/kb/en/

MySQL AB / Oracle Corporation. (2026). MySQL Reference Manual. Recuperado de https://dev.mysql.com/doc/

HeidiSQL Project. (2026). HeidiSQL User Documentation. Recuperado de https://www.heidisql.com/help.php

Apache Friends. (2026). XAMPP Community Resources. Recuperado de https://www.apachefriends.org/faq.html